

High Performance Computing

CS5013

Lab - 10

Student Details

Name: R Abinav

Roll No: ME23B1004

Date: March 20, 2026

Source Code

Listing 1: vector addition using mpi collective communication

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define N 1000000

int main(int argc, char *argv[])
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int local_n = N / size;
    double *a = NULL, *b = NULL, *c = NULL;
    double *local_a = (double*)malloc(local_n * sizeof(double));
    double *local_b = (double*)malloc(local_n * sizeof(double));
    double *local_c = (double*)malloc(local_n * sizeof(double));

    if(rank == 0)
    {
        a = (double*)malloc(N * sizeof(double));
        b = (double*)malloc(N * sizeof(double));
        c = (double*)malloc(N * sizeof(double));
        for(int i = 0; i < N; i++)
        {
            a[i] = (double)i * 1.5;
            b[i] = (double)i * 2.5;
        }
    }

    double start = MPI_Wtime();
```

```

MPI_Scatter(a, local_n, MPI_DOUBLE, local_a, local_n, MPI_DOUBLE
, 0, MPI_COMM_WORLD);
MPI_Scatter(b, local_n, MPI_DOUBLE, local_b, local_n, MPI_DOUBLE
, 0, MPI_COMM_WORLD);

for(int i = 0; i < local_n; i++)
    local_c[i] = local_a[i] + local_b[i];

MPI_Gather(local_c, local_n, MPI_DOUBLE, c, local_n, MPI_DOUBLE,
0, MPI_COMM_WORLD);

double end = MPI_Wtime();

if(rank == 0)
{
    printf("vector addition\n");
    printf("n=%d\n", N);
    printf("processes=%d\n", size);
    printf("time:%fseconds\n", end - start);
    printf("sample: %c[0] = %.2f, %c[1] = %.2f, %c[N-1] = %.2f\n",
        c[0], c[1], c[N-1]);
    free(a); free(b); free(c);
}

free(local_a); free(local_b); free(local_c);
MPI_Finalize();
return 0;
}

```

Output

```

abinav@Abinavs-MacBook-Air-97 lab-09 % mpicc -o mpi_vector_add mpi_vector_add.c
abinav@Abinavs-MacBook-Air-97 lab-09 % mpirun -np 8 ./mpi_vector_add
vector addition
n = 1000000
processes = 8
time: 0.021626 seconds
sample: c[0] = 0.00, c[1] = 4.00, c[N-1] = 3999996.00

```

Figure 1: output of mpi vector addition using collective communication for $n = 1000000$ with 8 processes

Source Code

Listing 2: vector multiplication using mpi collective communication

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define N 1000000

int main(int argc, char *argv[])
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int local_n = N / size;
    double *a = NULL, *b = NULL, *c = NULL;
    double *local_a = (double*)malloc(local_n * sizeof(double));
    double *local_b = (double*)malloc(local_n * sizeof(double));
    double *local_c = (double*)malloc(local_n * sizeof(double));

    if(rank == 0)
    {
        a = (double*)malloc(N * sizeof(double));
        b = (double*)malloc(N * sizeof(double));
        c = (double*)malloc(N * sizeof(double));
        for(int i = 0; i < N; i++)
        {
            a[i] = (double)(i + 1) * 1.5;
            b[i] = (double)(i + 1) * 2.5;
        }
    }

    double start = MPI_Wtime();

    MPI_Scatter(a, local_n, MPI_DOUBLE, local_a, local_n, MPI_DOUBLE,
        0, MPI_COMM_WORLD);
    MPI_Scatter(b, local_n, MPI_DOUBLE, local_b, local_n, MPI_DOUBLE,
        0, MPI_COMM_WORLD);

    for(int i = 0; i < local_n; i++)
        local_c[i] = local_a[i] * local_b[i];

    MPI_Gather(local_c, local_n, MPI_DOUBLE, c, local_n, MPI_DOUBLE,
        0, MPI_COMM_WORLD);
```

```

double end = MPI_Wtime();

if(rank == 0)
{
    printf("vector multiplication\n");
    printf("n=%d\n", N);
    printf("processes=%d\n", size);
    printf("time:%f seconds\n", end - start);
    printf("sample: c[0] = %.2f, c[1] = %.2f, c[N-1] = %.2f\n",
           c[0], c[1], c[N-1]);
    free(a); free(b); free(c);
}

free(local_a); free(local_b); free(local_c);
MPI_Finalize();
return 0;
}

```

Output

```

abinav@Abinavs-MacBook-Air-97 lab-09 % mpicc -o mpi_vector_mul mpi_vector_mul.c
abinav@Abinavs-MacBook-Air-97 lab-09 % mpirun -np 8 ./mpi_vector_mul
vector multiplication
n = 1000000
processes = 8
time: 0.024705 seconds
sample: c[0] = 3.75, c[1] = 15.00, c[N-1] = 3750000000000.00

```

Figure 2: output of mpi vector multiplication using collective communication for $n = 1000000$ with 8 processes